



**INSTITUTO SUPERIOR POLITÉCNICO
DE TECNOLOGIAS E CIÊNCIAS
ISPTEC**

Licenciatura em Engenharia Informática
Disciplina: Compiladores

MINI-COMPILADOR DA LINGUAGEM C
Analisador Léxico, Sintático e Semântico

Relatório do Projecto

Estudante: Lúcio Nguendelamba Marcial Vitorino
Número de estudante: 20232318
Docente: André Filemon

Luanda, Julho de 2026

Índice	2
1. Introdução	5
1.1 Objectivos	5
1.2 Estrutura do documento	5
2. Desenvolvimento	7
2.1 Especificação da linguagem em estudo	7
2.1.1 Construções suportadas	7
2.1.2 Conjunto de tokens	7
2.2 Fase 1 — Analisador Léxico (Scanner)	8
2.2.1 Fundamentos teóricos	8
2.2.2 Metodologia: despacho pelo primeiro caractere	8
2.2.3 Robustez do scanner	9
2.3 Fase 2 — Analisador Sintático (Parser)	9
2.3.1 Preparação da gramática	9
2.3.2 Metodologia: descida recursiva	10
2.3.3 Construção da AST	10
2.3.4 Recuperação de erros: modo pânico	11
2.4 Fase 3 — Analisador Semântico	11
2.4.1 Papel da fase semântica	11
2.4.2 Tabela de símbolos	12
2.4.3 Verificações semânticas implementadas	12
2.4.4 Integração das três fases	13
3. Conclusão	15
4. Referências Bibliográficas	16
Apêndice A — Manual do Programador	17
A.1 Objectivo e público-alvo	17
A.2 Organização do projecto	17
A.3 Fluxo de execução	17
A.4 Módulo Lexer	18
A.4.1 Estrutura t_token	18
A.4.2 Laço principal: tokenize()	18
A.4.3 Sub-autómatos de reconhecimento	18
A.4.4 Utilitários de suporte	19
A.5 Módulo AST	19
A.5.1 A estrutura ASTNode	19
A.5.2 Funções do módulo	20
A.5.3 Referência: disposição dos filhos por tipo de nó	20
A.6 Módulo Parser	21
A.6.1 Estado e navegação	21
A.6.2 Correspondência regra-função	22
A.6.3 A cadeia de precedência de expressões	22

A.6.4 Decisão declaração vs. expressão em blocos	23
A.6.5 Recuperação de erros: o módulo panic_mode	23
A.7 Módulo Semântico	23
A.7.1 A estrutura Symbol e a tabela	23
A.7.2 API da tabela	24
A.7.3 O percurso semântico	24
A.7.4 Como acrescentar uma nova verificação semântica	24
A.8 Exemplo guiado: do fonte à tabela de símbolos	25
A.8.1 Saída do lexer	25
A.8.2 Árvore construída pelo parser	25
A.8.3 Registo na tabela de símbolos	26
A.9 Catálogo de mensagens de erro do parser	26
A.10 Compilação e testes	27
A.11 Depuração	27
A.12 Guia de extensão	28
A.12.1 Acrescentar um novo token	28
A.12.2 Acrescentar uma nova construção sintática	28
A.12.3 Boas práticas do projecto	28
Apêndice B — Manual do Utilizador	29
B.1 Apresentação	29
B.2 Requisitos do sistema	29
B.3 Instalação	29
B.4 Utilização básica	30
B.5 A linguagem aceite	30
B.5.1 Directivas	30
B.5.2 Tipos e declarações	30
B.5.3 Funções	30
B.5.4 Estruturas de controlo	31
B.5.5 Expressões	31
B.6 Interpretação dos resultados	31
B.6.1 Programa válido	31
B.6.2 Formato das mensagens de erro sintático	31
B.6.3 Formato das mensagens de erro semântico	32
B.7 Exemplos completos de sessão	32
B.7.1 Validação de um programa com função	32
B.7.2 Erro na chamada de função	33
B.7.3 Redecaração no mesmo escopo	33
B.7.4 Programa com arrays, struct e ciclo	33
B.7.5 Validação de vários erros numa só execução	34
B.8 Catálogo de mensagens e como as resolver	34
B.8.1 Erros léxicos	34

B.8.2 Erros sintáticos	34
B.8.3 Erros semânticos	35
B.9 Utilização em scripts	35
B.10 Boas práticas para evitar erros	35
B.11 Limitações conhecidas	36
B.12 Resolução de problemas	36
B.13 Glossário	37

1. Introdução

Um compilador é um programa que traduz código escrito numa linguagem de programação de alto nível para uma representação que possa ser executada por uma máquina. O processo de compilação divide-se classicamente em duas grandes partes: o front-end, responsável pela análise do código-fonte, e o back-end, responsável pela síntese do código final. O front-end, por sua vez, organiza-se em três fases sequenciais: análise léxica, análise sintática e análise semântica.

O presente documento descreve a construção de um MINI-COMPILADOR para uma sublinguagem da linguagem C, desenvolvido integralmente em C no âmbito da disciplina de Compiladores. O projecto implementa as três fases do front-end: um analisador léxico (scanner) que converte o texto fonte numa sequência de tokens; um analisador sintático (parser) por descida recursiva que valida a estrutura gramatical e constrói uma Árvore Sintática Abstracta (AST); e um analisador semântico que, apoiado numa tabela de símbolos com escopos encadeados, verifica as regras de significado da linguagem.

A linguagem em estudo é um subconjunto representativo do C, escolhido por ser a linguagem de referência do curso e por permitir demonstrar, com clareza, os conceitos teóricos de Linguagens Formais e Autómatos aplicados à prática: autómatos finitos determinísticos no reconhecimento de lexemas, gramáticas livres de contexto na definição da sintaxe e regras de tipos e de escopo na semântica.

1.1 Objectivos

O objectivo geral do projecto é construir e defender o front-end completo de um compilador, culminando na fase de análise semântica. Os objectivos específicos são:

- Especificar a sublinguagem C em estudo, com o seu conjunto de tokens e a sua gramática livre de contexto;
- Implementar um analisador léxico modular baseado em autómatos finitos determinísticos e expressões regulares;
- Preparar a gramática (eliminação de ambiguidades e de recursividade à esquerda) e implementar um parser por descida recursiva com recuperação de erros em modo pânico;
- Construir a AST como estrutura intermédia entre a sintaxe e a semântica;
- Implementar a tabela de símbolos e o analisador semântico com as verificações exigidas: variável não declarada, redeclaração no mesmo escopo, incompatibilidade de tipos, verificação de argumentos de funções, atribuições incompatíveis e validação das condições das estruturas de controlo.

2. Desenvolvimento

2.1 Especificação da linguagem em estudo

A linguagem em estudo é uma sublinguagem do C que preserva as construções essenciais da linguagem original. A escolha do C justifica-se por três razões: é uma linguagem com gramática bem documentada; o seu conjunto de tokens exercita todos os casos clássicos de um scanner (identificadores, palavras reservadas, literais numéricos, strings, operadores compostos e comentários); e a sua semântica de tipos e escopos é suficientemente rica para as verificações exigidas.

2.1.1 Construções suportadas

- Directivas de pré-processamento: `#include <ficheiro>`, `#include "ficheiro"` e `#define NOME valor`;
- Tipos primitivos `int`, `float`, `char` e `void`, além de `struct` com corpo de campos e `typedef` para criação de aliases;
- Declarações de variáveis globais e locais, com ponteiros, arrays e inicialização, incluindo múltiplos declaradores na mesma instrução;
- Declarações e definições de funções, com listas de parâmetros tipados, protótipos e corpo em bloco;
- Estruturas de controlo: `if`, `while`, `do-while` e `for`, além de `return`;
- Expressões com a hierarquia completa de precedência do C: atribuição (simples e composta), lógicos, bit a bit, igualdade, relacionais, deslocamento, aditivos, multiplicativos, unários e pós-fixos (chamada de função, indexação de array, acesso a membro e incremento/decremento);
- Comentários de linha e de bloco, reconhecidos e descartados antes da análise sintática.

2.1.2 Conjunto de tokens

Cada token do sistema é representado pela estrutura `t_token`, que guarda o lexema (`value`), a classe do token (`type`), a linha de origem (`line`) e um ponteiro para o token seguinte, formando uma lista ligada. As principais classes de tokens são:

- `KEYWORD_*` — palavras reservadas reconhecidas (`int`, `float`, `char`, `void`, `return`, `if`, `else`, `while`, `for`, `do`, `struct`, `typedef`);
- `IDENTIFIER` — identificadores definidos pelo utilizador;
- `INT_LITERAL`, `FLOAT_LITERAL`, `CHAR_LITERAL`, `STRING_LITERAL` — literais;
- `ARITHMETIC_OP`, `ARITHMETIC_ASSIGNMENT_OP`, `COMPARISON_OPERATOR`, `LOGICAL_OPERATOR`, `BIT_OPERATOR`, `INCREMENT_OP`, `DECREMENT_OP`, `DIVISION_OP`, `ASSIGNMENT_OPERATOR` — operadores;

- DELIMITER — parênteses, chavetas, parênteses rectos, ponto e vírgula, vírgula e ponto;
- TOK_HASH, TOK_INCLUDE, TOK_DEFINE, TOK_NOME_FICHEIRO, TOK_LT, TOK_GT, TOK_DQUOTE — tokens específicos das directivas;
- COMMENT — comentários, preservados na lista mas ignorados pelo parser;
- TOK_EOF — marcador de fim de ficheiro, acrescentado no final da tokenização.

2.2 Fase 1 — Analisador Léxico (Scanner)

2.2.1 Fundamentos teóricos

A análise léxica assenta na teoria das linguagens regulares. Cada classe de token é descrita por uma expressão regular, e o reconhecimento é feito por um autómato finito determinístico (AFD) equivalente. Por exemplo, um identificador corresponde à expressão regular `[a-zA-Z][a-zA-Z0-9_]*`, um inteiro a `[0-9]+` e um real a `[0-9]+\.[0-9]+`. Durante o desenho do projecto, estes autómatos foram modelados na ferramenta JFLAP (ficheiro `compilador.jff`) antes da sua tradução para código.

Na implementação, cada AFD é materializado como uma função em C que percorre os caracteres da entrada mantendo um índice de posição — o índice funciona como o estado de leitura do autómato, e as condições sobre o caractere corrente correspondem às transições. Quando a função atinge um estado de aceitação (fim do padrão), o lexema reconhecido é isolado e convertido em token.

2.2.2 Metodologia: despacho pelo primeiro caractere

A função central é `tokenize(int fd)`, no ficheiro `Lexer/lexer.c`. Ela lê o ficheiro fonte linha a linha, através de `get_next_line()`, e percorre cada linha caractere a caractere. Em cada posição, o primeiro caractere determina qual sub-autómato deve ser accionado — uma estratégia conhecida como despacho por caractere inicial, que evita retrocessos e mantém o reconhecimento determinístico:

- Letra ou `_` — `process_identifier_keyWord()`: consome o maior prefixo alfanumérico e consulta a tabela de palavras reservadas para decidir entre `KEYWORD_*` e `IDENTIFIER` (princípio do maior lexema e prioridade de palavra-chave);
- Aspas ou plica — `process_identifier_string_char()`: reconhece literais de string e de caractere, com suporte a sequências de escape e detecção de aspas não fechadas;
- Dígito — `process_numeric_literal()`: consome a parte inteira e, se existir um ponto seguido de dígito, a parte fraccionária, distinguindo `INT_LITERAL` de `FLOAT_LITERAL`;
- `+` `-` `*` `%` — `process_arithmetic_operator()`: reconhece operadores simples, compostos (`+=`, `-=...`) e de incremento/decremento (`++`, `--`);

- `=` — `process_equals_sinal()`: distingue atribuição (`=`) de comparação (`==`);
- `<` `>` — `process_assignment_comparison_operator()`: reconhece relacionais (`<`, `<=`, `>`, `>=`) e deslocamentos (`<<`, `>>`, `<<=`, `>>=`);
- `&` `|` `!` `^` `~` — `process_logical_bitwise_operator()`: distingue lógicos (`&&`, `||`, `!`) de bit a bit;
- `/` — `process_division_operator_and_coment()`: decide entre divisão, `/=`, comentário de linha e comentário de bloco;
- `#` — `process_preprocessor_directive()`: tokeniza directivas `#include` e `#define` num fluxo estruturado de tokens (cardinal, nome da directiva, delimitadores e nome do ficheiro ou valor da macro);
- Delimitadores — `process_delimiter()`: emite um token `DELIMITER` por cada símbolo de pontuação estrutural.

Espaços, tabulações e quebras de linha são consumidos sem gerar tokens, funcionando apenas como separadores. Um contador de linha (`row`) é incrementado a cada linha lida e gravado em cada token, o que permite mensagens de erro localizadas nas fases seguintes. No final, é acrescentado o token `TOK_EOF`, que serve de sentinela para o parser.

2.2.3 Robustez do scanner

Duas protecções merecem destaque. Primeiro, o laço principal guarda a posição anterior (`old_i`) e, se nenhum sub-autómato consumir caracteres — por exemplo perante um símbolo desconhecido —, avança forçadamente uma posição, garantindo que o scanner nunca entra em ciclo infinito. Segundo, o reconhecimento de strings valida o fecho das aspas e reporta um erro léxico com a linha exacta quando o literal fica por fechar.

2.3 Fase 2 — Analisador Sintático (Parser)

2.3.1 Preparação da gramática

A sintaxe da linguagem foi formalizada numa gramática livre de contexto em notação BNF (documento `gramática_C.md`). Para que a gramática pudesse ser implementada por um parser descendente recursivo (classe LL), foi necessário prepará-la em duas frentes:

- Eliminação da recursividade à esquerda: as regras de expressões, naturalmente esquerdo-recursivas (por exemplo, `aditivo ::= aditivo + termo`), foram reescritas na forma equivalente com regras auxiliares à direita (`aditivo ::= multiplicativo aditivo_r`; `aditivo_r ::= + multiplicativo aditivo_r | ε`). Na implementação, estas regras auxiliares traduzem-se em ciclos `while` que acumulam operandos à esquerda, preservando a associatividade correcta;

- Eliminação de ambiguidades por estratificação de precedência: em vez de uma única regra de expressão ambígua, a gramática define um nível por precedência — atribuição, ou-lógico, e-lógico, ou-bit, xor-bit, e-bit, igualdade, deslocamento, relacional, aditivo, multiplicativo, unário, pós-fixado e primário — de forma que cada frase tem uma única árvore de derivação;
- Factorização à esquerda: regras com prefixo comum, como a declaração geral (que só se distingue entre função e variável depois do identificador), foram factorizadas com regras de cauda (`cauda_declaracao_geral`), permitindo decidir com um único token de antecipação.

2.3.2 Metodologia: descida recursiva

O parser foi implementado pela técnica de descida recursiva: cada não-terminal da gramática corresponde a uma função `parse_*`() que consome tokens e devolve o nó de AST correspondente. O estado do parser é mínimo — uma estrutura `Parser` com o ponteiro para a cabeça da lista de tokens e o ponteiro para o token corrente — e a antecipação é de um token (`LL(1)`), consultada através de `p->current` sem consumo.

A raiz é `parse_programa()`, que cria o nó `NODE_PROGRAMA` e itera declarações globais até `TOK_EOF`. Cada declaração global pode ser uma directiva de pré-processamento, um `typedef`, uma declaração de função ou uma declaração de variáveis. Dentro de funções, `parse_bloco()` processa a lista de itens do bloco, alternando entre declarações locais e instruções; a distinção entre declaração e expressão que começa por identificador (caso de tipos customizados via `typedef`) é resolvida com uma antecipação de dois tokens sobre a lista ligada.

As expressões seguem a cadeia de precedência descrita na gramática. Cada nível binário constrói nós `NODE_OP_BINARIO` com o operador em `op` e os operandos como filhos, associando à esquerda; a atribuição, pelo contrário, é implementada com recursão à direita (`parse_atribuicao` chama-se a si própria no lado direito), garantindo a associatividade à direita própria do C.

2.3.3 Construção da AST

A AST é a estrutura de saída do parser e a estrutura de entrada do analisador semântico. Cada nó (`ASTNode`) guarda o tipo do nó (`enum NodeType` com 33 variantes, de `NODE_PROGRAMA` a `NODE_VAZIO`), o texto associado (valor), o operador (`op`), a linha de origem, o nível de ponteiro, o número de dimensões de array e um vector dinâmico de filhos com crescimento geométrico (a capacidade duplica quando esgota). As funções `make_node()`, `make_folha()` e `add_filho()` encapsulam a criação e ligação de nós; `print_ast()` imprime a árvore indentada para depuração; `free_ast()` liberta a memória recursivamente.

Exemplos de formato: uma função é um `NODE_DECLARACAO_FUNCAO` cujo primeiro filho é o tipo de retorno, seguido dos parâmetros (`NODE_DECLARACAO_VARIAVEL`) e

do corpo (NODE_BLOCO) — ou de uma folha marcadora "proto" quando se trata de protótipo; um if é um NODE_IF com a condição e o corpo como filhos; um for é um NODE_FOR com exactamente quatro filhos (inicialização, condição, incremento e corpo), usando NODE_VAZIO nas posições omitidas para manter a aridade fixa.

2.3.4 Recuperação de erros: modo pânico

Um parser educacional deve reportar o máximo de erros numa única execução, em vez de abortar no primeiro. Para isso foi implementada a recuperação em modo pânico (panic mode recovery), no módulo parser/panic_mode.c. A ideia é: quando uma regra encontra um token inesperado, reporta o erro (mensagem, linha, token encontrado), incrementa o contador global de erros e descarta tokens até encontrar um token de sincronização — um ponto seguro a partir do qual a análise pode retomar.

Foram definidos cinco conjuntos de sincronização, adequados ao contexto do erro:

- SYNC_DECLARACAO — ponto e vírgula, chaveta de fecho, palavras-chave de tipo, cardinal e EOF: usado em erros ao nível das declarações globais;
- SYNC_BLOCO — chaveta de fecho e EOF: usado no fecho de blocos;
- SYNC_INSTRUCAO — ponto e vírgula, chaveta de fecho, palavras-chave de instrução e EOF: usado dentro de blocos;
- SYNC_EXPRESSAO — ponto e vírgula, parênteses e parênteses rectos de fecho, chaveta de fecho e EOF: usado dentro de expressões;
- SYNC_HEADER — parêntese de fecho, chaveta de abertura, ponto e vírgula: usado nos cabeçalhos de if/while/for/do-while.

A função parser_expect() centraliza o padrão: verifica o token esperado, consome-o em caso de sucesso e, em caso de falha, compõe a mensagem de erro, dispara syntax_error_recover() e sincroniza. Complementarmente, os laços de topo (parse_programa e parse_lista_itens_bloco) guardam o token corrente antes de cada tentativa e, se nenhum progresso for feito, consomem um token à força — uma blindagem anti-ciclo que garante terminação mesmo perante entradas patológicas.

Se, no final da análise sintática, o contador de erros for maior que zero, o programa reporta o total e termina antes da fase semântica, uma vez que não faz sentido validar significado sobre uma estrutura sintática inconsistente.

2.4 Fase 3 — Analisador Semântico

2.4.1 Papel da fase semântica

A análise semântica é a terceira fase do front-end e responde à pergunta: o programa, estando bem escrito, faz sentido? O analisador recebe a AST produzida pelo parser e valida cada uso de identificador e cada expressão com auxílio dos atributos registados na tabela de símbolos: nome, tipo de dado, categoria, escopo, endereço de memória simulado, tamanho em bytes, dimensões, valor de inicialização e, no caso das funções, lista de parâmetros e tipo de retorno.

2.4.2 Tabela de símbolos

A tabela de símbolos (`semantico/tabela_simbolos.c` e `.h`) é uma tabela de dispersão (hash table) com 211 posições e tratamento de colisões por encadeamento. A função de dispersão é a DJB2, escolhida pela boa distribuição sobre strings e pela simplicidade de implementação. Cada entrada é uma estrutura `Symbol` com os atributos exigidos no enunciado:

- `lexeme` e `token` — o nome do identificador e a classe do token de origem;
- `categoria` — variável, método (função), classe (struct) ou parâmetro;
- `tipo_dado` — `int`, `float`, `char`, `void`, `string` ou `struct`, com `tipo_customizado` para nomes de structs e typedefs;
- `escopo` — `global`, `classe` ou `método`, e `tipo_var` — `local`, `global` ou `atributo`;
- `endereco` — endereço de memória simulado, atribuído sequencialmente por escopo através do campo `next_endereco` da tabela;
- `tamanho` — tamanho em bytes do símbolo;
- `inicializado` e `valor` — indicador e valor textual da inicialização;
- `qtd_dimensoes` — número de dimensões, para arrays;
- `parametros` — lista ligada de parâmetros (tipo e nome) quando o símbolo é uma função, e `retorno` — o tipo de retorno;
- `escopo_membros` — sub-tabela de símbolos para os campos de uma struct.

O suporte a escopos é obtido por encadeamento de tabelas: cada tabela guarda um ponteiro `parent` para o escopo envolvente. Ao entrar numa função ou bloco cria-se uma nova tabela filha; ao sair, ela é destruída. A pesquisa `lookup_symbol()` procura primeiro no escopo corrente e sobe recursivamente até ao global — implementando a regra de visibilidade do C —, enquanto `lookup_symbol_current_scope()` restringe a pesquisa ao escopo corrente, exactamente o que é necessário para detectar redeclarações sem proibir o sobreamento (shadowing) legítimo de variáveis globais por locais.

2.4.3 Verificações semânticas implementadas

A função `analisar_semantica()` percorre a AST em profundidade, mantendo a tabela do escopo corrente, e aplica as verificações exigidas no enunciado:

- a) Variável não declarada: cada `NODE_IDENTIFICADOR` usado em expressão é procurado com `lookup_symbol()`; se não existir em nenhum escopo visível, é emitido erro semântico com a linha do uso;
- b) Redefinição no mesmo escopo: ao processar um `NODE_DECLARACAO_VARIAVEL`, `insert_symbol()` verifica se o lexema já existe no escopo corrente e recusa a inserção, gerando o erro de identificador duplicado;
- c) Incompatibilidade de tipos: a função `obter_tipo_expressao()` infere, de forma recursiva, o tipo de cada subexpressão (literals têm tipo directo; identificadores herdam o tipo registado na tabela; operações binárias promovem os operandos), e `tipos_compativeis()` codifica as regras de conversão aceites — atribuição de `int` a `float` e de `char` a `int` são permitidas (promoções implícitas do C); as restantes combinações geram erro;
- d) Verificação de argumentos de funções: numa `NODE_CHAMADA_FUNC`, o símbolo da função é localizado, e a lista de argumentos da chamada é confrontada com a lista de parâmetros registada: número de argumentos e compatibilidade de tipo posição a posição;
- e) Atribuição incompatível: em cada `NODE_ATRIBUICAO`, o tipo do lado direito é inferido e confrontado com o tipo do destino via `tipos_compativeis()`;
- f) Estruturas de controlo: as condições de `if`, `while`, `do-while` e `for` são verificadas quanto a produzirem um tipo utilizável como condição (`tipo_dado_eh_numerico()`), e as variáveis nelas usadas passam pelas mesmas verificações de declaração e tipo.

Os erros semânticos são acumulados num contador global, consultado no final por `obter_numero_erros_semanticos()`. Tal como na fase sintática, a análise não pára no primeiro erro: percorre toda a árvore e reporta o conjunto completo, terminando com o total de erros ou com a mensagem de sucesso.

2.4.4 Integração das três fases

O ponto de entrada `main.c` materializa o pipeline completo do front-end:

1. Abertura do ficheiro fonte passado como argumento;
2. `tokenize(fd)` produz a lista ligada de tokens;
3. `parser_init()` posiciona o parser e `parse_programa()` constrói a AST; havendo erros sintáticos, o programa reporta o total e termina;
4. `create_table(NULL)` cria a tabela de símbolos global;

5. `analisar_semantica(ast, symbol_table, ESCOPO_GLOBAL)` percorre a AST, preenche a tabela e valida as regras;
6. Havendo erros semânticos, o total é reportado; caso contrário é impresso "sucesso";
7. Toda a memória é libertada — tabela, AST e lista de tokens — e o ficheiro é fechado.

Esta separação estrita — o lexer não conhece gramática, o parser não conhece tipos, a semântica não conhece caracteres — é a decisão arquitectural central do projecto e reflecte a organização clássica descrita na literatura de compiladores.

3. Conclusão

O projecto atingiu o objectivo proposto: um front-end completo de compilador para uma sublinguagem do C, com as três fases implementadas, integradas e funcionais. O analisador léxico reconhece todas as classes de tokens da linguagem com localização por linha; o analisador sintático valida a estrutura do programa segundo uma gramática LL preparada para o efeito, constrói uma AST fiel e recupera de erros em modo pânico com conjuntos de sincronização por contexto; o analisador semântico, apoiado numa tabela de símbolos com dispersão e escopos encadeados, realiza as seis famílias de verificações exigidas no enunciado.

Do ponto de vista das aprendizagens, o projecto consolidou a ligação entre a teoria e a prática: os autómatos finitos deixaram de ser diagramas abstractos para se tornarem funções de reconhecimento; a eliminação de recursividade à esquerda e a estratificação de precedência revelaram-se pré-requisitos concretos, e não meros exercícios formais, para a descida recursiva funcionar; e a tabela de símbolos mostrou-se o verdadeiro elo entre a sintaxe e o significado. A recuperação de erros, em particular, exigiu um cuidado de engenharia que raramente é visível nos exemplos de manual: garantir progresso, escolher pontos de sincronização adequados a cada contexto e produzir mensagens úteis.

Como trabalho futuro, o projecto está preparado para crescer em três direcções naturais: alargar a linguagem (mais tipos, switch/case completo, enum, operador seta para ponteiros de struct); enriquecer a semântica (verificação de retorno em todos os caminhos, detecção de variáveis não utilizadas, avisos de conversões com perda); e iniciar o back-end, com geração de código intermédio de três endereços a partir da AST já anotada. A arquitectura modular adoptada — pastas independentes por fase, interfaces pequenas e estruturas de dados explícitas — foi pensada precisamente para tornar estas extensões localizadas e seguras.

4. Referências Bibliográficas

AHO, A. V.; LAM, M. S.; SETHI, R.; ULLMAN, J. D. Compiladores: Princípios, Técnicas e Ferramentas. 2.^a edição. São Paulo: Pearson Addison-Wesley, 2008.

APPEL, A. W. Modern Compiler Implementation in C. Cambridge: Cambridge University Press, 1998.

COOPER, K. D.; TORCZON, L. Engineering a Compiler. 2.^a edição. San Francisco: Morgan Kaufmann, 2011.

HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. Introdução à Teoria de Autômatos, Linguagens e Computação. 2.^a edição. Rio de Janeiro: Campus, 2002.

KERNIGHAN, B. W.; RITCHIE, D. M. The C Programming Language. 2.^a edição. Englewood Cliffs: Prentice Hall, 1988.

LOUDEN, K. C. Compiler Construction: Principles and Practice. Boston: PWS Publishing, 1997.

RODGER, S. H.; FINLEY, T. W. JFLAP: An Interactive Formal Languages and Automata Package. Sudbury: Jones & Bartlett, 2006.

Apêndice A — Manual do Programador

A.1 Objectivo e público-alvo

Este manual descreve a arquitectura interna, o fluxo de execução, as estruturas de dados e as convenções do Compilador-C, e destina-se a programadores que precisem de manter, estender ou depurar o projecto. Assume-se familiaridade com a linguagem C, com ponteiros e alocação dinâmica, e com os conceitos básicos de compiladores (tokens, gramáticas, árvores sintáticas e tabelas de símbolos). Para instruções de utilização do executável, consultar o Manual do Utilizador (Apêndice B).

A.2 Organização do projecto

O código está organizado em módulos com responsabilidades estritamente separadas:

- `main.c` — ponto de entrada; orquestra as três fases e a libertação de memória;
- `Lexer/` — reconhecimento de tokens: `lexer.c` (laço principal), `idenumKey.c` (identificadores e palavras-chave), `numeric.c` (literais numéricos), `stringchar.c` (strings, chars, divisão e comentários), `operators.c` (operadores e delimitadores), `processor_directive.c` (directivas) e `token.h` (estrutura `t_token` e protótipos);
- `parser/` — análise sintática: `init.c`, `programa.c`, `declaracao_global.c`, `statements.c`, `expressao.c`, `tipo.c`, `helpers.c`, `panic_mode.c/.h` e `parser.h`;
- `ast/` — estrutura da árvore sintática: `ast.h` e `ast.c`;
- `semantico/` — tabela de símbolos e análise semântica: `tabela_simbolos.h/.c` e `analizador_semantico.c`;
- `utils/` — funções auxiliares: `get_next_line.c`, `ft_strjoin.c` e `token_operating.c` (criação, inserção, impressão e libertação de tokens).

Nota de manutenção: o ficheiro `parser/parser.c` contém uma versão inicial e simplificada de várias regras (versão legada) e não faz parte do build actual. As implementações em vigor são as dos módulos individuais listados acima. Ao editar o parser, confirmar sempre que se está a alterar o módulo compilado, e não o legado.

A.3 Fluxo de execução

O ponto de entrada é `main.c`. O fluxo, do texto fonte ao veredicto final, é o seguinte:

1. Validar os argumentos e abrir o ficheiro fonte com `open()`;
2. `tokenize(fd)` lê o ficheiro e devolve a lista ligada de tokens, terminada por `TOK_EOF`;
3. `parser_init(&p, tokens)` posiciona o parser no primeiro token;
4. `parse_programa(&p)` constrói a AST completa;

5. Se `parser_error_count() > 0`, o programa reporta o total de erros e termina — a fase semântica não corre sobre estrutura inválida;
6. `create_table(NULL)` cria a tabela de símbolos global (sem pai);
7. `analisar_semantica(ast, symbol_table, ESCOPO_GLOBAL)` percorre a AST, regista símbolos e valida as regras semânticas;
8. Se `obter_numero_erros_semanticos() > 0`, o total é reportado; caso contrário imprime-se "sucesso";
9. `free_table()`, `free_ast()`, `token_clear_list()` e `close()` libertam todos os recursos.

A.4 Módulo Lexer

A.4.1 A estrutura `t_token`

```
typedef struct s_token {
    char *value;           /* lexema reconhecido           */
    char *type;            /* classe do token (string)    */
    int line;              /* linha de origem             */
    struct s_token *next; /* proximo token (lista ligada) */
} t_token;
```

A classe do token é representada como string (por exemplo "IDENTIFIER", "INT_LITERAL"), o que simplifica a depuração e a comparação nos módulos do parser via `strcmp()`. Toda a criação passa por `token_create()` (aloca e copia `value` e `type`) e a inserção por `token_insert_back()`; a função `generate_token()` combina as duas operações e é a via preferencial nos sub-autómatos.

A.4.2 O laço principal: `tokenize()`

`tokenize(int fd)` lê o ficheiro linha a linha com `get_next_line()` e percorre cada linha com um índice `i`. O primeiro caractere de cada posição decide o sub-autómato a accionar (despacho por caractere inicial). Cada função `process_*`() recebe o ponteiro para a posição corrente da linha, a lista de tokens e a linha actual, e devolve o ponteiro para a primeira posição após o lexema consumido; o laço converte esse ponteiro de volta em índice. Três invariantes devem ser preservadas por qualquer alteração:

- Cada `process_*`() devolve sempre uma posição igual ou posterior à recebida — nunca retrocede;
- A blindagem `if (i <= old_i) i = old_i + 1` garante progresso perante caracteres não reconhecidos: qualquer novo sub-autómato deve manter esta protecção intacta;
- O contador `row` incrementa exactamente uma vez por linha lida, e todos os tokens da linha usam esse valor.

A.4.3 Sub-autómatos de reconhecimento

`process_identifier_keyWord()` consome o maior prefixo de caracteres alfanuméricos e underscore, e delega em `determine_token_type()` a decisão palavra-chave/identificador. A

tabela de palavras-chave é um par de arrays estáticos paralelos (lexema → classe); para acrescentar uma palavra reservada basta acrescentar uma entrada em cada array, mantendo as posições alinhadas.

`process_numeric_literal()` consome a parte inteira e só aceita o ponto decimal se este for seguido de dígito — assim, "9.metodo" não é lido como real. Devolve `INT_LITERAL` ou `FLOAT_LITERAL`.

`process_identifier_string_char()` trata strings e chars com a mesma máquina, parametrizada pela aspa de abertura. Sequências de escape (\") são saltadas aos pares para não fechar o literal prematuramente. Se a linha terminar sem aspa de fecho, é impresso um erro léxico com a linha e a função devolve a posição corrente para evitar ciclo.

`process_arithmetic_operator()`, `process_equals_sinal()`, `process_assignment_comparison_operator()` e `process_logical_bitwise_operator()` seguem todos o mesmo padrão: espreitar o segundo caractere para decidir entre operador simples e composto, isolar o lexema com `strndup()` e emitir o token com a classe adequada.

`process_division_operator_and_coment()` é o caso mais rico: distingue `"/"` (comentário até ao fim da linha), `"/*...*/"` (comentário de bloco, incluindo o caso não fechado), `"/="` e a divisão simples.

`process_preprocessor_directive()` tokeniza a directiva em unidades estruturadas: `TOK_HASH`, depois `TOK_INCLUDE` ou `TOK_DEFINE`, e em seguida os componentes — para o include, `TOK_LT/TOK_GT` ou `TOK_DQUOTE` em redor de `TOK_NOME_FICHEIRO`; para o define, o `IDENTIFIER` da macro e o literal opcional do valor. Directivas desconhecidas geram `TOK_DIRECTIVE` com o resto da linha em `TOK_DIRECTIVE_REST`. No fim, a função consome sempre até ao fim da linha, garantindo que a directiva nunca contamina a linha seguinte.

A.4.4 Utilitários de suporte

`get_next_line()` implementa a leitura linha a linha sobre `read()`, com buffer estático de 1024 bytes que preserva o estado entre chamadas. `token_print_list()` imprime a lista completa de tokens (value, type, line) e é a primeira ferramenta de depuração do lexer; `token_clear_list()` liberta a lista no fim.

A.5 Módulo AST

A.5.1 A estrutura ASTNode

```
typedef struct ASTNode {
    NodeType type;           /* tipo do nó (enum)           */
    char *valor;             /* texto associado           */
    char *op;                /* operador, quando aplicável */
    int linha;               /* linha de origem           */
    struct ASTNode **filhos; /* vector dinâmico de filhos */
    int n_filhos;            /* número de filhos          */
    int cap_filhos;          /* capacidade alocada        */
    int pointer_level;       /* níveis de ponteiro (*)    */
    int dimensions;          /* dimensões de array        */
} ASTNode;
```

O enum `NodeType` tem 33 variantes que cobrem o programa (`NODE_PROGRAMA`, `NODE_LISTA_DECL_GLOBAIS`), as directivas (`NODE_DIRETIVA_INCLUDE`, `NODE_DIRETIVA_DEFINE`), as declarações (`NODE_DECLARACAO_TYPEDEF`, `NODE_DECLARACAO_VARIAVEL`, `NODE_DECLARACAO_FUNCAO`), as instruções (`NODE_BLOCO`, `NODE_IF`, `NODE_WHILE`, `NODE_DO_WHILE`, `NODE_FOR`, `NODE_SWITCH`, `NODE_CASE`, `NODE_RETURN`, `NODE_BREAK`, `NODE_CONTINUE`, `NODE_INSTR_EXPR`), as expressões (`NODE_ATRIBUICAO`, `NODE_OP_BINARIO`, `NODE_OP_UNARIO`, `NODE_POS_FIXO`, `NODE_CHAMADA_FUNC`, `NODE_ACESSO_ARRAY`, `NODE_ACESSO_MEMBRO`) e as folhas (`NODE_IDENTIFICADOR`, `NODE_ESPECIFICADOR_TIPO`, `NODE_NOME_FICHEIRO`, literais e `NODE_VAZIO`).

A.5.2 Funções do módulo

- `make_node(type, linha)` — cria um nó vazio com todos os campos inicializados;
- `make_folha(type, valor, linha)` — cria um nó folha, duplicando o texto com `strdup()`;
- `add_filho(pai, filho)` — anexa um filho ao fim do vector; quando a capacidade esgota, duplica-a com `realloc()` (crescimento geométrico, custo amortizado constante);
- `print_ast(n, depth)` — imprime a árvore em profundidade, com indentação de dois espaços por nível, mostrando o tipo do nó, a linha, valor, op, nível de ponteiro e dimensões quando existem;
- `free_ast(n)` — liberta recursivamente os filhos, o vector, as strings e o próprio nó.

Regra de propriedade da memória: cada nó é dono das strings `valor` e `op` e do seu vector de filhos. Quem cria um nó e o anexa a um pai transfere a propriedade — a libertação é sempre feita, uma única vez, por `free_ast()` a partir da raiz. Ao escrever novas regras de parsing, nunca libertar um nó já anexado.

A.5.3 Referência: disposição dos filhos por tipo de nó

Ao percorrer a AST (por exemplo, no analisador semântico), é essencial conhecer a ordem convencionada dos filhos de cada nó. A referência é:

- `NODE_PROGRAMA` — um único filho: a lista de declarações globais;
- `NODE_DIRETIVA_INCLUDE` — um filho `NODE_NOME_FICHEIRO`, com o nome do ficheiro em valor e a origem em op ("system" para <...>, "local" para "...");
- `NODE_DIRETIVA_DEFINE` — o nome da macro em valor; filho opcional com o literal ou identificador do valor;
- `NODE_DECLARACAO_TYPEDEF` — o alias em valor; primeiro filho é o tipo original;
- `NODE_DECLARACAO_FUNCAO` — o nome em valor e `pointer_level` no campo próprio; filho 0 é o tipo de retorno; seguem-se os parâmetros (cada um `NODE_DECLARACAO_VARIAVEL` com o seu tipo como filho); o último filho é o corpo (`NODE_BLOCO`) ou a folha marcadora "proto" nos protótipos;
- `NODE_DECLARACAO_VARIAVEL` — o nome em valor, `pointer_level` e `dimensions` nos campos próprios; filho 0 é o tipo; filho 1, quando existe, é a expressão de inicialização; declaradores adicionais da mesma instrução local são anexados como filhos seguintes;
- `NODE_BLOCO` — um filho: a lista de itens do bloco;
- `NODE_IF` / `NODE_WHILE` — filho 0 é a condição, filho 1 o corpo;
- `NODE_DO_WHILE` — filho 0 é o corpo, filho 1 a condição (ordem sintática do do-while);
- `NODE_FOR` — aridade fixa de quatro: inicialização, condição, incremento e corpo, com `NODE_VAZIO` nas posições omitidas;
- `NODE_RETURN` — zero ou um filho (a expressão devolvida);
- `NODE_INSTR_EXPR` — um filho: a expressão da instrução;
- `NODE_ATRIBUICAO` e `NODE_OP_BINARIO` — o operador em op; filho 0 é o lado esquerdo, filho 1 o direito;
- `NODE_OP_UNARIO` e `NODE_POS_FIXO` — o operador em op e a subexpressão como filho único;
- `NODE_CHAMADA_FUNC` — filho 0 é a expressão chamada (tipicamente o identificador da função); filho 1, quando existe, é a lista de argumentos;
- `NODE_ACESSO_ARRAY` — filho 0 é a expressão indexada, filho 1 o índice;
- `NODE_ACESSO_MEMBRO` — o nome do membro em valor e a expressão base como filho.

A.6 Módulo Parser

A.6.1 Estado e navegação

```
typedef struct Parser {  
    t_token *token_stream; /* cabeca da lista de tokens */  
    t_token *current;      /* token corrente */  
} Parser;
```

A navegação é feita por quatro primitivas em `init.c`: `parser_next_token()` (consome e devolve o token corrente), `parser_peek_value()` e `parser_peek_type()` (antecipação sem consumo) e o par `parser_consume_if_value()` / `parser_consume_if_type()` (consumo condicional, devolvendo 1 em caso de sucesso). Todo o parser está escrito sobre estas primitivas; qualquer regra nova deve usá-las em vez de manipular `p->current` directamente.

A.6.2 Correspondência regra-função

O parser é descendente recursivo: cada não-terminal da gramática corresponde a uma função `parse_*`() que devolve o nó de AST construído (ou NULL em caso de falha). A tabela de correspondência principal é:

- `parse_programa()` (`programa.c`) — raiz; itera declarações globais até TOK_EOF com blindagem anti-ciclo;
- `parse_declaracao_global()` (`declaracao_global.c`) — despacha entre directivas (`#include`, `#define`), `typedef`, função e variável global;
- `parse_especificador_tipo()` (`tipo.c`) — tipos primitivos, identificadores de tipos customizados e struct com corpo de campos;
- `parse_bloco()`, `parse_lista_itens_bloco()`, `parse_item_bloco()` (`statements.c`) — blocos e a alternância declaração local/instrução;
- `parse_instrucao_if/while/for/do/return()` (`statements.c`) — estruturas de controlo;
- `parse_expressao()` e a cadeia de precedência (`expressao.c`) — expressões;
- `parse_asteriscos()`, `parse_sufixo_array_opcional()`, `parse_parametros_opcionais()` (`helpers.c`) — ponteiros, arrays e parâmetros.

A.6.3 A cadeia de precedência de expressões

A ordem real de chamada, do nível mais fraco ao mais forte, é:

```
parse_expressao -> parse_atribuicao -> parse_logico_ou ->  
parse_logico_e -> parse_bit_ou -> parse_bit_xor -> parse_bit_e ->  
parse_igualdade -> parse_bit_shift -> parse_relacional ->  
parse_aditivo -> parse_multiplicativo -> parse_unario ->  
parse_pos_fixo -> parse_primario
```

Cada nível binário segue o mesmo esqueleto: analisar o nível imediatamente mais forte, e enquanto o token corrente for um operador deste nível, consumir, analisar o operando direito e envolver ambos num `NODE_OP_BINARIO` — o que produz associatividade à

esquerda. A exceção é `parse_atribuicao()`, que chama a si própria no operando direito, produzindo a associatividade à direita do C ($a = b = c$). Para introduzir um novo nível de precedência: criar a função com o esqueleto acima, encaixá-la na cadeia (a função do nível mais fraco passa a chamá-la) e declará-la em `parser.h`.

`parse_pos_fixo()` aplica, em ciclo, os operadores pós-fixos sobre o resultado do primário: indexação (`NODE_ACESSO_ARRAY`), chamada de função (`NODE_CHAMADA_FUNC`, com `parse_argumentos_opcionais()` a construir a lista de argumentos), acesso a membro com ponto (`NODE_ACESSO_MEMBRO`) e incremento/decremento pós-fixos (`NODE_POS_FIXO`). `parse_unario()` trata `!`, `-` (simétrico), `&` (endereço) e `*` (desreferência) com recursão à direita.

A.6.4 Decisão declaração vs. expressão em blocos

Dentro de um bloco, um item que começa por palavra-chave de tipo é claramente uma declaração; mas um item que começa por `IDENTIFIER` tanto pode ser uma declaração com tipo customizado (Ponto `p1;`) como uma expressão (`p1 = 3;`). `parse_item_bloco()` resolve o dilema com antecipação directa sobre a lista ligada: salta os asteriscos a partir de `p->current->next` e verifica se o token seguinte é outro `IDENTIFIER` — se for, trata como declaração; caso contrário, como expressão. Esta é uma decisão LL(2) implementada manualmente, e deve ser tida em conta se a sintaxe de declarações for alargada.

A.6.5 Recuperação de erros: o módulo `panic_mode`

O módulo define os cinco conjuntos de sincronização (`SYNC_DECLARACAO`, `SYNC_BLOCO`, `SYNC_INSTRUCAO`, `SYNC_EXPRESSAO`, `SYNC_HEADER`) como arrays de strings terminados em `NULL`, que casam tanto com o `value` como com o `type` do token — o que permite usar `;"` e `"KEYWORD_INT"` no mesmo conjunto. As três funções públicas são:

- `recover(p, sync_set)` — descarta tokens até encontrar um membro do conjunto (ou `TOK_EOF`), deixando o token âncora como corrente;
- `syntax_error_recover(p, msg, sync_set)` — imprime a mensagem de erro no `stderr` com a linha e o token encontrado, incrementa o contador global e sincroniza;
- `parser_expect(p, valor, msg, sync_set)` — o padrão consome-ou-entra-em-pânico: devolve 1 se o token esperado estava presente e foi consumido; caso contrário compõe a mensagem, dispara o erro e devolve 0.

Convenção de uso: pontos obrigatórios da gramática (fecho de parênteses, ponto e vírgula) devem passar por `parser_expect()` com o conjunto de sincronização do contexto; pontos opcionais usam `parser_consume_if_value()`. O contador de erros é lido por `parser_error_count()` e reiniciado por `parser_reset_error_count()` em `parser_init()`.

A.7 Módulo Semântico

A.7.1 A estrutura Symbol e a tabela

```
typedef struct Symbol {
    char *token; char *lexeme; int linha;
    CategoriaSimbolo categoria; /* VARIAVEL|METODO|CLASSE|PARAMETRO */
    TipoDado tipo_dado; /* INT|FLOAT|CHAR|VOID|STRING|STRUCT */
    TipoVariavel tipo_var; /* LOCAL|GLOBAL|ATRIBUTO */
    EscopoSimbolo escopo; /* GLOBAL|CLASSE|METODO */
    char *valor; long endereco; int tamanho;
    bool inicializado; int qtd_dimensoes;
    ParamNode *parametros; char *modificadores;
    TipoDado retorno; char *tipo_customizado;
    struct SymbolTable *escopo_membros;
    struct Symbol *next; /* colisao na hash */
} Symbol;
```

A tabela (SymbolTable) tem 211 posições (HASH_SIZE, número primo, para melhor dispersão), um ponteiro parent para o escopo envolvente e um contador next_endereco para o endereçamento simulado. A função de dispersão é a DJB2 ($\text{hash} = \text{hash} * 33 + c$), aplicada ao lexema e reduzida módulo 211. As colisões são resolvidas por encadeamento, com inserção à cabeça da lista.

A.7.2 API da tabela

- `create_table(parent)` — cria uma tabela ligada ao escopo pai (NULL para o global);
- `insert_symbol(st, sym)` — insere no escopo corrente; devolve false se o lexema já existir nesse escopo (é este retorno que materializa o erro de redeclaração); atribui o endereço simulado sequencialmente segundo o tamanho do símbolo;
- `lookup_symbol(st, lexeme)` — pesquisa recursiva do escopo corrente até ao global (regra de visibilidade); devolve NULL se o símbolo não for visível (erro de variável não declarada);
- `lookup_symbol_current_scope(st, lexeme)` — pesquisa restrita ao escopo corrente, usada para permitir sombreamento legítimo;
- `tipos_compatíveis(destino, origem)` — regras de compatibilidade: tipos iguais, `int`→`float` e `char`→`int` são aceites; tudo o resto é incompatível;
- `tipo_dado_eh_numerico(t)` — verdadeiro para `int`, `float` e `char`; usado na validação de condições e operações aritméticas;
- `print_table(st)` — imprime a tabela formatada (lexema, token, linha, categoria, tipo, escopo, endereço, dimensões, tamanho, inicialização e valor) — a ferramenta principal de depuração da fase semântica;
- `free_table(st)` — liberta todas as entradas, incluindo listas de parâmetros e sub-tabelas de membros de structs.

A.7.3 O percurso semântico

`analisar_semantica(node, st, escopo)` percorre a AST em profundidade. Nos nós de declaração, constrói o Symbol correspondente (nome, tipo, categoria, dimensões, tamanho, inicialização) e tenta inseri-lo; nos nós de uso (identificadores em expressões, chamadas de função, atribuições, condições de controlo), consulta a tabela e valida tipos com apoio de `obter_tipo_expressao()`, que infere recursivamente o tipo de qualquer subexpressão. Ao entrar no corpo de uma função, cria-se uma tabela filha com `create_table(tabela_global)` onde são registados os parâmetros; ao sair, a tabela local é destruída. Os erros são impressos com a linha do nó e acumulados no contador consultado por `obter_numero_erros_semanticos()`.

A.7.4 Como acrescentar uma nova verificação semântica

1. Identificar o(s) tipo(s) de nó da AST onde a regra se manifesta (por exemplo, `NODE_RETURN` para validar o tipo de retorno);
2. No ponto do percurso onde esse nó é visitado, obter os símbolos e tipos envolvidos via `lookup_symbol()` e `obter_tipo_expressao()`;
3. Codificar a condição de violação e, quando violada, imprimir a mensagem com a linha do nó e incrementar o contador de erros;
4. Nunca abortar: registar o erro e continuar o percurso, para que todos os erros do programa sejam reportados numa única execução;
5. Acrescentar um ficheiro de teste que exercite a regra nos dois sentidos (caso válido e caso inválido).

A.8 Exemplo guiado: do fonte à tabela de símbolos

Para consolidar a leitura dos módulos, segue-se o percurso completo de um pequeno programa pelas três fases.

```
int main(void) {  
    int x = 5;  
    return x;  
}
```

A.8.1 Saída do lexer

A lista de tokens produzida por `tokenize()` (visível com `token_print_list()`) é, na essência:

value= int,	type= KEYWORD_INT,	line= 1
value= main,	type= IDENTIFIER,	line= 1
value= (,	type= DELIMITER,	line= 1
value= void,	type= KEYWORD_VOID,	line= 1
value=),	type= DELIMITER,	line= 1
value= {,	type= DELIMITER,	line= 1
value= int,	type= KEYWORD_INT,	line= 2
value= x,	type= IDENTIFIER,	line= 2
value= =,	type= ASSIGNMENT_OPERATOR,	line= 2
value= 5,	type= INT_LITERAL,	line= 2

value= ;,	type= DELIMITER,	line= 2
value= return,	type= KEYWORD_RETURN,	line= 3
value= x,	type= IDENTIFIER,	line= 3
value= ;,	type= DELIMITER,	line= 3
value= },	type= DELIMITER,	line= 4
value= EOF,	type= TOK_EOF,	line= 5

A.8.2 Árvore construída pelo parser

parse_programa() devolve, para esta entrada, a árvore (impressa por print_ast()):

```
[NODE_PROGRAMA] (L0)
  [NODE_LISTA_DECL_GLOBAIS] (L0)
    [NODE_DECLARACAO_FUNCAO] (L1) val="main"
      [NODE_ESPECIFICADOR_TIPO] (L1) val="int"
      [NODE_ESPECIFICADOR_TIPO] (L1) val="void"
      [NODE_BLOCO] (L1)
        [NODE_LISTA_DECL_GLOBAIS] (L0)
          [NODE_DECLARACAO_VARIAVEL] (L2) val="x"
            [NODE_ESPECIFICADOR_TIPO] (L2) val="int"
            [NODE_LIT_INTEIRO] (L2) val="5"
          [NODE_RETURN] (L3)
            [NODE_IDENTIFICADOR] (L3) val="x"
```

Lê-se directamente a convenção de filhos da secção A.5.3: a função main tem o tipo de retorno como primeiro filho, o marcador de parâmetros void a seguir e o bloco no fim; a variável x tem o tipo como filho 0 e a inicialização como filho 1.

A.8.3 Registo na tabela de símbolos

O percurso semântico regista main como símbolo de categoria METODO no escopo global (retorno int, lista de parâmetros vazia por ser void) e cria uma tabela filha para o corpo da função, onde x é registado como VARIAVEL de tipo int, escopo LOCAL, tamanho 4 bytes, endereço simulado 0, inicializada com o valor "5" e zero dimensões. O uso de x no return é resolvido por lookup_symbol() na tabela local, e o tipo da expressão devolvida (int) é compatível com o retorno declarado. Sem violações, o contador de erros permanece a zero e o main.c imprime "sucesso".

A.9 Catálogo de mensagens de erro do parser

As mensagens de erro sintático estão espalhadas pelos módulos, mas seguem todas o mesmo formato de saída (linha, descrição, token encontrado). As principais famílias e a sua origem:

- Declarações globais (declaracao_global.c) — "Falta o token '>' no include do sistema", "Falta a aspas de fecho no include local", "Esperado '<' ou aspas duplas após '#include'", "Esperado identificador após '#define'", "Esperado um tipo valido após 'typedef'", "Esperado o nome do alias no typedef", "Falta o parêntese de fecho na declaracao da funcao", "Esperado corpo de funcao ou ';' no prototipo", "Esperado ';' no final da declaracao global", "Esperado identificador após ';' na declaracao global";

- Blocos e instruções (statements.c) — "Falta a chaveta de fecho '}' para fechar o bloco", "Instrução ou item de bloco inválido.", "Esperado identificador após o especificador de tipo", "Expressão de inicialização inválida", "Esperado ';' após declaração de variável", "Esperado ':' após a expressão.";
- Cabeçalhos de controlo (statements.c) — "Falta o parêntese de abertura após o 'if'", "Falta o parêntese de fecho da condição do 'if'", equivalentes para while e do-while, e a sequência do for: "Esperado '(' após a palavra-chave 'for'", "Esperado ';' após a inicialização do 'for'", "Esperado ';' após a condição do 'for'", "Esperado ')' após o cabeçalho do 'for'"; ainda "Esperado 'while' após o corpo do comando 'do'" e "Falta o ':' no final da instrução do-while";
- Expressões (expressao.c) — "Expressão inválida ou vazia dentro dos parênteses", "Falta o parêntese de fecho ')' na expressão", "Falta o parêntese de fecho ')' na chamada da função", "Argumento inválido ou vazio na chamada da função", "Esperado argumento após a vírgula";
- Structs (tipo.c) — "Esperado especificador de tipo no campo da struct", "Esperado identificador após ',' na struct", "Esperado ';' após campo da struct", "Esperado '}' no fecho da struct".

Ao acrescentar mensagens novas, manter o estilo existente: frase curta, em português, orientada ao símbolo em falta ou inesperado, e sempre disparada através de `syntax_error_recover()` ou `parser_expect()` para que o contador e a sincronização funcionem de forma uniforme.

A.10 Compilação e testes

A compilação de todos os módulos pode ser feita directamente com o gcc (num só comando) ou através do Makefile do projecto:

```
gcc -Wall -Wextra -g -I. -ILexer -Iparser -Iast -Iutils -Isemantico \
-o compilador main.c \
  Lexer/lexer.c Lexer/idenumKey.c Lexer/operators.c \
  Lexer/stringchar.c Lexer/numeric.c Lexer/processor_directive.c \
  utils/get_next_line.c utils/ft_strjoin.c utils/token_operating.c \
  ast/ast.c parser/init.c parser/panic_mode.c parser/expressao.c \
  parser/tipo.c parser/helpers.c parser/declaracao_global.c \
  parser/programa.c parser/statements.c \
  semantico/tabela_simbolos.c semantico/analizador_semantico.c
```

As flags `-Wall` `-Wextra` devem ser mantidas sempre activas — vários defeitos históricos do projecto foram apanhados por avisos do compilador. A flag `-g` permite depuração simbólica com `gdb` e diagnóstico de memória com `valgrind`. A execução sobre um ficheiro de teste é:

```
./compilador testes/teste_identificadores.c
```

A.11 Depuração

A estratégia de depuração recomendada segue a ordem das fases:

- Suspeita no léxico — descomentar a chamada a `token_print_list(tokens)` em `main.c` e confirmar que a lista de tokens corresponde ao esperado (lexema, classe e linha);
- Suspeita no sintático — descomentar `print_ast(ast, 0)` e comparar a árvore impressa com a estrutura do programa fonte; verificar em particular a ordem e o número de filhos dos nós;
- Suspeita no semântico — chamar `print_table()` nos pontos relevantes do percurso para inspeccionar os símbolos registados, os seus tipos, escopos e endereços;
- Erros de memória — compilar com `-g` e correr `valgrind --leak-check=full --track-origins=yes ./compilador teste.c`; o projecto liberta toda a memória nos caminhos normais e de erro, e qualquer alteração deve preservar essa propriedade;
- Inspeção passo a passo — `gdb ./compilador`, com breakpoints nas funções `parse_*`() ou em `analisa_semantica()`.

A.12 Guia de extensão

A.12.1 Acrescentar um novo token

1. Definir o reconhecimento: acrescentar uma função `process_*`() no Lexer (ou estender uma existente) e ligá-la ao despacho em `lexer.c`;
2. Escolher a classe do token (string) e documentá-la em `token.h`;
3. Se o token participa na gramática, actualizar as regras do parser que o consomem;
4. Acrescentar um caso de teste léxico.

A.12.2 Acrescentar uma nova construção sintática

1. Formalizar a regra na gramática (`gramatica_C.md`), garantindo que não introduz recursividade à esquerda nem ambiguidade;
2. Implementar a função `parse_*`() no módulo adequado, devolvendo o nó de AST; se necessário, criar um novo `NodeType` em `ast.h` e o respectivo caso em `print_ast()`;
3. Definir o comportamento de erro: que `parser_expect()` usar e com que conjunto de sincronização;
4. Ligar a nova função ao ponto de despacho correcto (`parse_instrucao()`, `parse_declaracao_global()`, etc.);
5. Estender o analisador semântico para visitar o novo nó, nem que seja apenas para percorrer os filhos.

A.12.3 Boas práticas do projecto

- Funções pequenas e especializadas: cada função reconhece uma única construção;
- Nunca misturar fases: o lexer não conhece gramática, o parser não conhece tipos, a semântica não lê caracteres;
- Verificar sempre `p->current` antes de o desreferenciar — a lista termina, e vários defeitos históricos vieram de desreferências após o fim;
- Toda a string guardada em estruturas é duplicada com `strdup()/strndup()`; a propriedade pertence à estrutura e é libertada uma única vez;
- Mensagens de erro sempre com linha e com o token encontrado — são a interface do compilador com o utilizador.

Apêndice B — Manual do Utilizador

B.1 Apresentação

O Compilador-C é uma ferramenta de linha de comandos que analisa ficheiros escritos numa sublinguagem da linguagem C e verifica se estão correctos em três níveis: léxico (as palavras e símbolos são válidos), sintático (a estrutura do programa respeita a gramática) e semântico (o programa faz sentido — as variáveis estão declaradas, os tipos são compatíveis, as funções são chamadas correctamente). A ferramenta não gera código executável: o seu resultado é um veredicto de validação, acompanhado, quando necessário, de mensagens de erro com a linha onde o problema ocorre.

Este manual destina-se a qualquer pessoa que pretenda utilizar o compilador para validar programas, sem necessidade de conhecer o seu funcionamento interno. Para detalhes de arquitectura e manutenção, consultar o Manual do Programador (Apêndice A).

B.2 Requisitos do sistema

- Sistema operativo Linux, ou Windows com WSL (Windows Subsystem for Linux);
- Compilador GCC instalado (para construir a ferramenta a partir do código-fonte);
- Utilitário make (opcional, mas recomendado);
- Um terminal com acesso à linha de comandos.

Para verificar se o GCC está instalado, executar `gcc --version` no terminal. Em sistemas Debian/Ubuntu, a instalação faz-se com `sudo apt install build-essential`.

B.3 Instalação

O compilador é distribuído em código-fonte. Os passos de instalação são:

1. Copiar (ou extrair) a pasta do projecto para o computador;
2. Abrir um terminal dentro da pasta do projecto (onde está o Makefile);
3. Executar o comando make. A construção gera o executável na raiz do projecto;
4. Confirmar a instalação executando o programa sem argumentos — deve aparecer a mensagem de utilização.

```
$ make
$ ./compilador
Usage: ./compilador <source-file.c>
```

Comandos úteis do Makefile: `make` constrói o executável; `make clean` remove os ficheiros objecto; `make fclean` remove também o executável. Depois de qualquer alteração ao código-fonte, executar `make` novamente.

No Windows com WSL, os mesmos comandos são executados dentro do terminal WSL. Os ficheiros do Windows ficam acessíveis em `/mnt/c/...`

B.4 Utilização básica

O compilador recebe exactamente um argumento: o caminho do ficheiro fonte a analisar.

```
$ ./compilador programa.c
```

Existem três desfechos possíveis:

- Sucesso — o programa é válido nas três fases; é impressa a palavra `sucesso` e o programa termina com código 0;
- Erros sintáticos — a estrutura do programa viola a gramática; cada erro é reportado com a linha e o token encontrado, seguido do total de erros, e o programa termina com código 1 sem chegar à fase semântica;
- Erros semânticos — a estrutura está correcta, mas há violações de significado (variáveis não declaradas, tipos incompatíveis, etc.); cada erro é reportado com a linha, seguido do total, e o programa termina com código 1.

O código de saída (0 ou 1) permite integrar o compilador em scripts: por exemplo, validar automaticamente todos os ficheiros de uma pasta.

B.5 A linguagem aceite

O compilador aceita um subconjunto da linguagem C. Em termos práticos, pode escrever-se:

B.5.1 Directivas

```
#include <stdio.h>
#include "meuheader.h"
#define MAX 100
#define PI 3.14
```

B.5.2 Tipos e declarações

Os tipos disponíveis são `int`, `float`, `char` e `void`, além de structs e aliases criados com `typedef`. As declarações admitem ponteiros, arrays, inicialização e múltiplas variáveis na mesma linha:

```
int contador = 0;
float media, soma = 0.0;
char letra = 'A';
int *ptr;
int tabela[10];

struct Ponto {
    int x;
    int y;
};

typedef int Inteiro;
```

B.5.3 Funções

```
int soma(int a, int b) {
```

```

    return a + b;
}

void imprimir(void); /* prototipo tambem e aceite */

```

B.5.4 Estruturas de controlo

```

if (x > 0) { y = 1; } else { y = 2; }
while (i < 10) { i = i + 1; }
do { i = i - 1; } while (i > 0);
for (i = 0; i < 10; i = i + 1) { total = total + i; }

```

B.5.5 Expressões

São aceites todos os operadores habituais do C, com a precedência correcta: aritméticos (+, -, *, /, %), de atribuição (=, +=, -=, *=, /=, %=), relacionais (<, >, <=, >=), de igualdade (==, !=), lógicos (&&, ||, !), bit a bit (&, |, ^, <<, >>), incremento e decremento (++, --), chamadas de função, indexação de arrays (a[i]) e acesso a membros de struct (p.x). Comentários de linha (//) e de bloco (/* */) são ignorados.

B.6 Interpretação dos resultados

B.6.1 Programa válido

Para o ficheiro seguinte:

```

int main(void) {
    int x = 5;
    return x;
}

```

a execução produz:

```

$ ./compilador exemplo.c
sucesso

```

B.6.2 Formato das mensagens de erro sintático

Os erros sintáticos seguem o formato: linha do erro, descrição do problema e o token efectivamente encontrado nesse ponto. Exemplo — falta de ponto e vírgula:

```

int main(void) {
    int x = 5
    return x;
}

ERRO : linha 3: Esperado ';' apos declaracao de variavel
        encontrado: tipo="KEYWORD_RETURN" val="return"
Total de erros: 1

```

Repare-se que o erro é detectado na linha 3 (onde surge o token inesperado), embora a causa esteja na linha 2. Esta é uma característica normal dos compiladores: perante uma omissão, o erro manifesta-se no primeiro símbolo que deixa de fazer sentido. Recomenda-se olhar também para a linha imediatamente anterior à indicada.

O compilador não pára no primeiro erro: recupera e continua a análise, reportando os erros seguintes. Em cascatas de erros, corrigir primeiro o primeiro da lista e voltar a compilar — os restantes podem ser consequência dele.

B.6.3 Formato das mensagens de erro semântico

Os erros semânticos indicam a linha e a regra violada. Exemplo — variável não declarada e tipos incompatíveis:

```
int main(void) {
    int x = 5;
    y = 10;          /* y nunca foi declarada      */
    char c = 3.14;   /* float nao cabe num char */
    return x;
}
```

produz mensagens que identificam a variável não declarada na linha 3 e a incompatibilidade de tipos na linha 4, terminando com o total de erros. As verificações realizadas nesta fase são:

- Uso de variáveis não declaradas;
- Declaração da mesma variável duas vezes no mesmo escopo;
- Operações entre tipos incompatíveis;
- Chamadas de função com número ou tipos de argumentos errados;
- Atribuições de um tipo incompatível a uma variável;
- Condições inválidas em if, while, do-while e for.

Conversões implícitas seguras são aceites sem erro: atribuir um int a um float e um char a um int é permitido, tal como na linguagem C.

B.7 Exemplos completos de sessão

B.7.1 Validação de um programa com função

```
#include <stdio.h>
#define LIMITE 10

int soma(int a, int b) {
    return a + b;
}

int main(void) {
    int total = 0;
    int i;
    for (i = 0; i < LIMITE; i = i + 1) {
        total = soma(total, i);
    }
    return total;
}

$ ./compilador exemplo_soma.c
sucesso
```

B.7.2 Erro na chamada de função

Se a chamada da função anterior for alterada para `soma(total)` — com um só argumento — a fase semântica detecta a divergência entre a chamada e a declaração (dois parâmetros esperados) e reporta o erro com a linha da chamada.

B.7.3 Redeclaração no mesmo escopo

```
int main(void) {
    int x = 1;
    int x = 2;    /* redeclaracao invalida */
    return x;
}
```

A segunda declaração de `x` é recusada pela tabela de símbolos e o erro é reportado na linha 3. Note-se que declarar uma variável local com o mesmo nome de uma global é permitido (sombreamento) — o erro só ocorre quando as duas declarações estão no mesmo escopo.

B.7.4 Programa com arrays, struct e ciclo

O exemplo seguinte exercita, num só ficheiro, structs, typedef, arrays, ponteiros e as quatro estruturas de controlo aceites:

```
#include <stdio.h>
#define TAMANHO 5

struct Ponto {
    int x;
    int y;
};

typedef int Inteiro;

int somar_vector(int v[], int n) {
    Inteiro total = 0;
    int i;
    for (i = 0; i < n; i = i + 1) {
        total = total + v[i];
    }
    return total;
}

int main(void) {
    int valores[TAMANHO];
    int i = 0;
    while (i < TAMANHO) {
        valores[i] = i * 2;
        i = i + 1;
    }
    if (somar_vector(valores, TAMANHO) > 10) {
        return 1;
    }
    return 0;
}

$ ./compilador exemplo_completo.c
sucesso
```

B.7.5 Validação de vários erros numa só execução

Uma vantagem prática do compilador é reportar todos os erros de uma vez, em lugar de parar no primeiro. Para o ficheiro:

```
int main(void) {  
    int a = 1  
    int b = 2;  
    c = a + b;  
    return 0;  
}
```

a análise sintática detecta a falta do ponto e vírgula da linha 2 e recupera; corrigido esse erro e recompilando, a análise semântica reporta ainda o uso da variável `c` não declarada na linha 4. O ciclo natural de trabalho é: compilar, corrigir os erros indicados de cima para baixo e compilar novamente até obter "sucesso".

B.8 Catálogo de mensagens e como as resolver

A lista seguinte reúne as mensagens mais frequentes, o seu significado e a correcção típica.

B.8.1 Erros léxicos

- "Erro léxico: aspas não fechadas na linha N" — uma string (") ou um carácter (') aberto nessa linha nunca foi fechado. Acrescentar a aspa/plica em falta; atenção às sequências de escape, que não contam como fecho.

B.8.2 Erros sintáticos

- "Esperado ';' ..." (após declaração, expressão, typedef, etc.) — falta o ponto e vírgula no fim da instrução indicada; frequentemente a causa está na linha anterior à reportada;
- "Falta o parêntese de abertura/fecho ..." — os parênteses de um cabeçalho `if/while/for/do-while` ou de uma expressão estão desequilibrados; contar aberturas e fechos na linha indicada;
- "Falta a chaveta de fecho '}' para fechar o bloco" — um bloco aberto com `{` não foi fechado; típico de funções ou ciclos com chavetas a menos no fim do ficheiro;
- "Esperado identificador após o especificador de tipo" — depois de um tipo (`int`, `float`...) não surgiu um nome de variável válido; verificar se o nome não é uma palavra reservada;
- "Esperado 'while' após o corpo do comando 'do'" — a estrutura `do-while` está incompleta: ao corpo `do` tem de seguir-se `while` (condição);
- "Argumento inválido..." / "Esperado argumento após a vírgula" — a lista de argumentos de uma chamada tem uma vírgula a mais ou um argumento vazio, como em `soma(a, )`;
- "Esperado '<' ou aspas duplas após '#include'" — a directiva `include` não indica o ficheiro entre `<...>` nem entre `"..."`;

- "Instrução ou item de bloco inválido." — o compilador não conseguiu interpretar a linha nem como declaração nem como instrução; frequentemente resulta de um erro anterior ou de uma construção fora do subconjunto suportado (secção B.9).

B.8.3 Erros semânticos

- Variável não declarada — o identificador é usado sem declaração prévia visível; declarar a variável antes do uso, no escopo adequado, e confirmar que não há erro de escrita no nome (maiúsculas e minúsculas contam);
- Identificador já declarado neste escopo — duas declarações com o mesmo nome no mesmo bloco; renomear uma delas ou remover a duplicada;
- Tipos incompatíveis (em operação, atribuição ou argumento) — confirmar os tipos declarados dos operandos; lembrar que `int→float` e `char→int` são aceites, mas por exemplo `float→char` não;
- Número ou tipo de argumentos errado na chamada — comparar a chamada com a declaração da função: mesma quantidade de argumentos, tipos compatíveis posição a posição;
- Condição inválida em estrutura de controlo — a expressão dentro de `if/while/for/do-while` deve produzir um valor numérico; verificar variáveis não declaradas ou tipos não numéricos dentro da condição.

B.9 Utilização em scripts

Como o compilador devolve o código de saída 0 em caso de sucesso e 1 em caso de erro, pode ser usado em scripts de validação automática. O exemplo seguinte valida todos os ficheiros `.c` de uma pasta de testes e resume o resultado:

```
#!/bin/bash
for f in testes/*.c; do
    if ./compilador "$f" > /dev/null 2>&1; then
        echo "OK    $f"
    else
        echo "ERRO  $f"
    fi
done
```

Para guardar as mensagens de erro de uma execução num ficheiro, redireccionar a saída de erros: `./compilador programa.c 2> erros.txt`. As mensagens de erro são escritas no canal de erros (`stderr`) e o veredicto de sucesso no canal normal (`stdout`), pelo que podem ser separados.

B.10 Boas práticas para evitar erros

- Terminar cada instrução com ponto e vírgula antes de mudar de linha — é, de longe, o erro sintático mais comum;
- Fechar imediatamente cada símbolo que se abre: escrever `()` e `{}` aos pares e só depois preencher o interior;
- Declarar as variáveis no início do bloco onde são usadas, o que torna os erros de "variável não declarada" fáceis de localizar;
- Manter nomes consistentes: o compilador distingue maiúsculas de minúsculas, pelo que `Total` e `total` são variáveis diferentes;
- Compilar com frequência, a cada pequena alteração — quanto menos código novo entre duas compilações, mais fácil é localizar o erro;
- Perante muitos erros de uma vez, corrigir sempre o primeiro da lista e recompilar antes de atacar os seguintes.

B.11 Limitações conhecidas

Por se tratar de um subconjunto do C, algumas construções da linguagem completa não são suportadas nesta versão:

- Tipos `double`, `long`, `short`, `unsigned` e qualificadores como `const` e `static`;
- As instruções `switch/case`, `break` e `continue` (previstas na AST, mas não activadas na gramática desta versão);
- O operador seta (`->`) para acesso a membros através de ponteiro, o operador condicional (`?:`) e `sizeof`;
- `enum`, `union` e inicializadores de agregados entre chavetas;
- Expansão real de macros e inclusão efectiva de ficheiros: as directivas `#include` e `#define` são reconhecidas e validadas, mas os ficheiros não são incluídos nem as macros substituídas no texto;
- Geração de código: a ferramenta valida o programa mas não produz executável.

Escrever fora deste subconjunto resulta em erros léxicos ou sintáticos — não significa que o programa esteja errado em C, apenas que usa construções fora da linguagem em estudo.

B.12 Resolução de problemas

- "Usage: ./compilador <source-file.c>" — o programa foi executado sem argumento ou com mais de um; indicar exactamente um ficheiro;
- "Error opening file" — o caminho indicado não existe ou não há permissões de leitura; confirmar o caminho com `ls` e as permissões com `ls -l`;

- "command not found" ao executar ./compilador — o executável ainda não foi construído; correr make primeiro;
- Erros estranhos em cascata — corrigir o primeiro erro reportado e recompilar; muitos erros subsequentes desaparecem;
- "Erro léxico: aspas não fechadas na linha N" — existe uma string ou um carácter sem a aspa/plica de fecho nessa linha;
- O programa parece correcto em C mas é rejeitado — verificar a lista de limitações da secção B.11: a construção usada pode estar fora do subconjunto suportado.

Persistindo um comportamento inesperado, registar o ficheiro de entrada e a saída completa do terminal e reportar ao responsável pelo projecto — essa informação é suficiente para reproduzir e diagnosticar o problema.

B.13 Glossário

- Token — a menor unidade com significado do código-fonte: uma palavra reservada, um nome, um número, um operador ou um símbolo de pontuação;
- Lexema — o texto concreto de um token tal como aparece no ficheiro (por exemplo, o lexema "contador" de um token identificador);
- Análise léxica — a fase que transforma o texto do ficheiro em tokens e detecta símbolos inválidos;
- Análise sintática — a fase que verifica se a sequência de tokens respeita a gramática da linguagem (estrutura de declarações, instruções e expressões);
- Análise semântica — a fase que verifica o significado: declaração e visibilidade de variáveis, compatibilidade de tipos e correcção das chamadas de função;
- AST (árvore sintática abstracta) — a representação interna, em árvore, da estrutura do programa, construída pela análise sintática;
- Tabela de símbolos — a estrutura interna onde o compilador regista cada nome declarado com os seus atributos (tipo, escopo, tamanho, endereço, inicialização);
- Escopo — a região do programa onde um nome é visível: global (todo o ficheiro) ou local (dentro de uma função ou bloco);
- Declaração — a introdução de um nome com o seu tipo (int x;); Protótipo — a declaração de uma função sem corpo, terminada em ponto e vírgula;
- Directiva de pré-processamento — linha iniciada por #, como #include ou #define;
- Código de saída — o número devolvido ao terminal quando o programa termina: 0 indica sucesso e 1 indica que foram encontrados erros.